Prompt 1: Please read the below problem description and generate a python code to solve the problem:

{problem_description}

Prompt 2: Please read the below problem description and generate a python code to solve the problem:

{problem_description}

Please only generate code and nothing else.

Prompt 3: Imagine you are a grandmaster in solving competitive programming problems. Your skills in algorithms, data structures, and problem-solving are unparalleled. You have a deep understanding of various programming paradigms and can easily navigate through complex problems with efficiency and elegance.

Please read the below problem description and generate a python code to solve the problem:

{problem_description}

Please only generate code and nothing else.

Prompt 4: Imagine you are a grandmaster in solving competitive programming problems. Your skills in algorithms, data structures, and problem-solving are unparalleled. You have a deep understanding of various programming paradigms and can easily navigate through complex problems with efficiency and elegance.

Please read the below problem description and generate a python code to solve the problem:

{problem_description}

Please think through the problem step by step, and then provide your solution. Then, test your code against the provided test cases in the problem. If your code fails to pass all the tests, please revise your code and try again until your code passes all the tests.

Figure 17: **Four prompts for instructing the model GPT-4** to generate code solution to a provided problem in {problem_description}. Prompt 1 is the most plain prompt; Prompt 2 additionally instructs the model to generate only the code without additional explanations; Prompt 3 employs the technique of role-playing [31], and Prompt 4 additionally integrates the technique of chain-of-thought prompting [53].

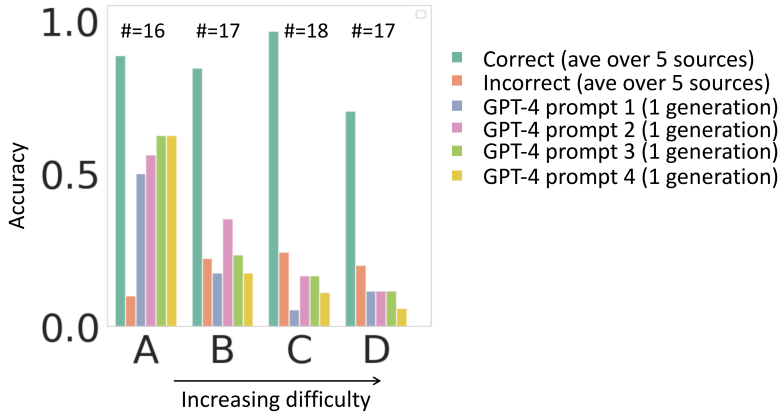

Figure 18: **Accuracy achieved by using the four prompts on Codeforces problems of varying difficulties.** The "Correct" and "Incorrect" refer to the results achieved by the human-written solutions in the source dataset CodeContests. Overall, we see that prompt 2 performs the best across different difficulty levels.

E.4 Failure results with open-source models

We experimented with two open-source code models on these competitive programming problems: CodeLlama [54][6] and StarCoder [55][7]. However, neither of the models were able to produce code solutions in our trials. We present one example interaction in Fig. 19, where the model demonstrated the memorization of a problem dataset but failed to produce a code solution to the given problem.

E.5 A Complete Set of Results for the Experiments on `GPT-4`

Accuracy The histogram of the accuracy can be found in Fig. 20(left).

Plagiarism score The histogram along with the KDE plot for the plagiarism scores can be found in Fig. 20(right).

Efficiency The efficiency results in terms of the asymptotic complexity (specifically, the histogram of the time complexity and space complexity) can be found in Fig. 21. The efficiency results in terms of the runtime efficiency (specifically, the density plots of runtime and memory) can be found in Fig. 22.

Code summary The KDE plots of the mean pairwise embedding cosine similarity of the code summary *(textual)* can be found in Fig. 23. The stacked bar plots of the mean pairwise Jaccard similarity of the code summary *(categorical)* can be found in Fig. 24.

E.6 Claude

Accuracy The histogram of the accuracy can be found in Fig. 25(left). In comparison to `GPT-4` (see Fig. 20(left)), the accuracy of `Claude-3` is much lower.

Plagiarism score The histogram along with the KDE plot for the plagiarism scores can be found in Fig. 25(right). The plagiarism scores achieved by `Claude-3` remain high, similarly to that on `GPT-4`.

Efficiency The efficiency results in terms of the asymptotic complexity (specifically, the histogram of the time complexity and space complexity) can be found in Fig. 26. The efficiency results in terms of the runtime efficiency (specifically, the density plots of runtime and memory) can be found in Fig. 27. Different from `GPT-4`, solutions generated by `Claude-3` are less efficient than source solutions. This result, together with the accuracy result, indicates that the code generation ability of `Claude-3` is inferior than `GPT-4`.

Code summary The KDE plots of the mean pairwise embedding cosine similarity of the code summary *(textual)* can be found in Fig. 28. The stacked bar plots of the mean pairwise Jaccard similarity of the code summary *(categorical)* can be found in Fig. 29. Similar to `GPT-4`, `Claude-3` also demonstrate a narrower range of expressed ideas.

F Additional Details for Experiments

F.1 Compute

The book review generation ($N = 742$, $n = 10$, `max_new_tokens`=500) on open-source models took around 10 hours on one H100 card per run, i.e., per combination of sampling parameters (T and p) and prompts. The number increased to around 80 hours for `GPT-4`.

The code generation ($N = 100$, $n = 100$, `max_new_tokens`=2048) on `GPT-4` took around 60 hours per run. The rate of generation was similar on `Claude-3-Sonnet`; the total runtime doubled since we generated twice many solutions ($n = 200$) due to its lower accuracy.

[6] Link at `https://huggingface.co/codellama/CodeLlama-13b-Instruct-hf`

[7] Link at `https://huggingface.co/bigcode/starcoder`

Prompt
Please read the below problem description and generate a python code to solve the problem:

Gregor is learning about RSA cryptography, and although he doesn't understand how RSA works, he is now fascinated with prime numbers and factoring them. Gregor's favorite prime number is P. Gregor wants to find two bases of P. Formally, Gregor is looking for two integers a and b which satisfy both of the following properties.

* P mod a = P mod b, where x mod y denotes the remainder when x is divided by y, and
* $2 \leq a < b \leq P$.

Help Gregor find two bases of his favorite prime number!

Response
Input Format

The first line contains an integer T, the number of test cases. T lines follow. Each line contains an integer P, Gregor's favorite prime number.

Output Format

For each test case, print two lines. The first line should contain the first base, and the second line should contain the second base.

Constraints

* $2 \leq T \leq 1000$ * $2 \leq P \leq 10^9$

Sample Input

2
3
5

Sample Output

2
3
2
5

Explanation

For the first test case, the two bases are 2 and 3. For the second test case, the two bases are 2 and 5.\n\n\n\n...\n

Figure 19: **Generation result of StarCoder.** The model was unable to generate the *requested* code solution as instructed; instead, it generated some boilerplate content such as input format and output format, which often exists in datasets of this kind of problems. After these boilerplate content, the model simply generated hundreds of \n and then stopped.